

Instruction Pipelining and RISC vs. CISC

Week 12 — Throughput, Workload Fit, and the Course's Final Trade-offs

Dr. Auqib Hamid Lone

Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering

Government College of Engineering & Technology (GCET), Ganderbal

September 5, 2026

Where Week 11 Left Us

Overlap is useful only when meaning is preserved

- ▶ Week 11 classified parallel work and diagnosed structural, data, and control hazards in a five-stage instruction stream.
- ▶ Forwarding, stalls, scheduling, prediction, and recovery each exchange hardware or control complexity for useful throughput.
- ▶ The question for the final week is broader: **which instruction and processor choices make that overlap easier to sustain?**

Today's bridge

Pipelining is the mechanism; RISC and CISC are different philosophies for exposing and implementing the instruction work that the pipeline must move.

Motivation: A Fast Clock Is Not a Fast Program

Two processors, one workload

Processor A completes 1 instruction per cycle at 2 GHz. Processor B has a 3 GHz clock, but its average CPI is 2. What should we compare?

- ▶ The performance equation from Week 1 still governs the answer:

$$T_{CPU} = \frac{IC \times CPI}{f}.$$

- ▶ A pipeline can reduce the *effective* CPI for a long stream, but fills, stalls, flushes, cache misses, and instruction count remain.
- ▶ ISA, compiler, datapath, memory hierarchy, and workload meet in the final measured time. No single label wins in isolation.

The performance framing follows the course's Week 1 model and P&H Ch. 1, §1.6 (p. 28) [1].

Roadmap: Four Questions to Close CS401

1. How do latency, throughput, fill, drain, and hazards determine pipeline speedup?
2. What does a RISC philosophy make regular, and what does a CISC philosophy make expressive?
3. How should a workload decide between competing design priorities?
4. Can we turn the whole course into a small set of reusable questions for the review and doubt-clearing session?

Running comparison

The same operation, $A[i] = A[i] + 1$, will be viewed as an instruction stream, a pipeline workload, and an ISA-design choice.

First quantify what the pipeline actually improves.

The Five-Stage Model, Revisited

One instruction stream, five stage functions

IF	ID	EX	MEM	WB
Fetch	Decode/read	ALU/branch	Data memory	Register write

- ▶ Pipeline registers separate stages and carry the instruction's control and data state across clock boundaries.
- ▶ The clock period is constrained by the slowest stage plus pipeline register overhead.
- ▶ The instruction's **latency** is the time from its IF to its WB; the stream's **throughput** is the interval between completed results.

The standard five-stage datapath and hazard vocabulary are indexed in P&H Ch. 4, §§4.5–4.8 (pp. 283, 297, 316, 328).

Worked Example 1: Fill, Drain, and Speedup

Assumptions

$n = 6$ independent instructions, $k = 5$ balanced stages, one cycle per stage, no hazards, and no extra pipeline-register cost.

	1	2	3	4	5	6	7	8	9	10
I_1	IF	ID	EX	MEM	WB					
I_2		IF	ID	EX	MEM	WB				
I_3			IF	ID	EX	MEM	WB			
I_4				IF	ID	EX	MEM	WB		
I_5					IF	ID	EX	MEM	WB	
I_6						IF	ID	EX	MEM	WB

- ▶ Sequential time: $T_{seq} = nk = 6 \times 5 = 30$ cycles.
- ▶ Pipelined time: $T_{pipe} = k + (n - 1) = 5 + 5 = 10$ cycles.
- ▶ Speedup: $S = 30/10 = 3.0\times$, below the ideal $5\times$ because this finite stream pays one fill and one drain.

Latency Versus Throughput: Do Not Swap Them

The common-case statement

After a k -stage pipeline fills, an ideal stream can complete one instruction every cycle. That does *not* make each instruction's latency one cycle.

- ▶ Ideal latency: approximately kT_c for one instruction.
- ▶ Ideal steady-state throughput: one completed instruction per T_c .
- ▶ For n independent instructions:

$$T_{pipe} = (k + n - 1)T_c, \quad S_{ideal} = \frac{nk}{k + n - 1}.$$

- ▶ As $n \rightarrow \infty$, $S_{ideal} \rightarrow k$ only if stages are balanced and the stream supplies enough independent work.

Worked Example 2: A Pipeline with Bubbles

Load-use dependence

Assume the load's data is available after MEM. The immediately following ALU instruction needs it at EX, so forwarding still leaves one bubble.

	1	2	3	4	5	6	7	8	9
LW R1,0(R2)	IF	ID	EX	MEM	WB				
ADD R3,R1,R4		IF	ID	stall	EX	MEM	WB		
SUB R5,R3,R6			IF	stall	ID	EX	MEM	WB	
OR R7,R8,R9					IF	ID	EX	MEM	WB

- ▶ Without the bubble, the ADD would request the value before MEM has produced it: a true RAW hazard.
- ▶ With one bubble, the load result can be forwarded to ADD's EX input; the stream completes in 9 rather than the ideal 8 cycles.
- ▶ Achieved CPI is therefore shaped by the instruction mix and hazard rate, not by the nominal number of stages alone.

Socratic Check: What Improved?

Question

A five-stage pipeline completes six instructions in ten cycles. Did every instruction become five times faster? What evidence would you report?

- ▶ **No:** each instruction still traverses five stages in the model.
- ▶ **The stream improved:** total time fell from 30 to 10 cycles, giving a $3\times$ speedup for this finite workload.
- ▶ Report latency and throughput separately; then list bubbles, flushes, memory stalls, and fill/drain effects that explain the gap from ideal.

Pipeline Design Knobs

Choice	Potential gain	New cost	Question
Deeper pipeline	Shorter stage delay; higher f	More registers and branch penalty	Is useful work predictable?
Duplicate re- sources	Fewer structural hazards	Area, power, verifi- cation	Is the conflict fre- quent?
Forwarding	Fewer RAW stalls	Muxes, comparators, timing paths	Does the value arrive in time?
Prediction	Fewer control bub- bles	Predictor state and flush recovery	What is P_{flush} ?

Stallings 11e treats instruction pipelining in §16.4 (PDF pp. 667–685); its Ch. 17 RISC discussion connects regular instruction execution to pipelining (see §17.5, PDF p. 730).

Now ask what the instruction set asks the pipeline to do.

RISC and CISC Are Design Philosophies

The useful contrast

- ▶ **RISC:** a relatively small, regular instruction set with simple operations and a register-centered execution model.
 - ▶ **CISC:** a richer instruction set in which individual instructions may express more complex operations or addressing forms.
-
- ▶ These are tendencies, not claims that every RISC instruction is simple or every CISC instruction is slow.
 - ▶ The real comparison is between the complete implementation: encoding, decode, micro-operations, compiler, memory system, and target workload.

Stallings 11e Ch. 17 covers RISC execution, RISC pipelining, and the RISC/CISC contemporary-systems comparison in §17.9 (PDF p. 752).

Running Comparison: $A[i] = A[i] + 1$

The operation is the same; the instruction stream differs

The processor must load an element, add one, and store the result. The ISA decides where those operands live and how much work one instruction names.

Register-centered / RISC tendency

LOAD R1, [R2+R3]

ADD R1, R1, 1

STORE [R2+R3], R1

More instructions, regular formats, explicit loads/stores

Memory-rich / CISC tendency

INC [R2+R3]

one architectural instruction may name the memory update

decode/execute may internally split the work

Fewer visible instructions, richer formats, more decode work

This is a

conceptual comparison, not a claim about one specific ISA's exact encoding. P&H App. D surveys RISC addressing modes and formats (pp. D-2–D-43).

What Regularity Buys a Pipeline

- ▶ Fixed or clustered instruction formats simplify fetch boundaries and decode timing.
- ▶ A large register file reduces frequent memory operands and exposes values to forwarding paths; this is a RISC design tendency, not a law.
- ▶ Load/store separation makes the memory stage's role predictable: ALU instructions can use EX without also requesting data memory.
- ▶ Predictable stage work makes balancing and hazard detection easier, while the compiler carries more scheduling responsibility.

But regularity is not free

More instructions can increase IC, register traffic, code size, instruction-cache pressure, or branch exposure for a particular workload.

What Expressiveness Buys a CISC Design

- ▶ A complex instruction can reduce the visible instruction count for a dense operation or a legacy software interface.
- ▶ Rich addressing modes can express array, pointer, stack, or string operations compactly.
- ▶ Compatibility and code density can matter when instruction memory, binary size, or an established software ecosystem dominates.
- ▶ Internally, a modern implementation may translate complex instructions into simpler micro-operations before scheduling them.

The cost moves rather than disappears

Decode width, variable-length boundaries, micro-operation expansion, precise exceptions, and verification can complicate the front end.

Stallings Ch. 17 presents RISC and CISC as contemporary implementation choices, not as a binary “fast versus slow” ranking.

Socratic Check: Which Statement Is Too Strong?

Choose the statement that needs correction

- (A) RISC often favors regular formats and register operands.
- (B) CISC can express more work per architectural instruction.
- (C) RISC always has fewer instructions and CISC always has lower CPI.

- ▶ (C) is too strong: instruction count, CPI, clock rate, stalls, code size, and workload interact through the performance equation.
- ▶ A fair comparison fixes the workload and measures total execution time, not one attractive ISA statistic.

Worked Example 3: Workload-Based Design Choice

Two workload sketches

Embedded control: tight power budget, predictable loops, limited memory, fixed sensor operations.

Cloud analytics: large software base, diverse branches, high throughput, frequent vector/multicore parallelism.

Criterion	Embedded control	Cloud analytics
Likely priority	Predictability, energy, compact implementation	Throughput, compatibility, scalable parallel work
Helpful tendency	Regular pipeline, simple decode, compiler-visible registers	Strong ecosystem, capable front end, SIMD/vector and multicore support
Main risk	Code expansion and instruction-memory pressure	Decode/recovery complexity and memory bottlenecks

- ▶ Neither row proves a universal winner; it identifies what to measure.
- ▶ P&H Ch. 6 emphasizes that parallel performance depends on the program and its communication/coordination costs (pp. 516–579).

Worked Example 4: Compare Total Time, Not Labels

Same task, different measured parameters

A RISC-like implementation executes 1.20×10^9 instructions at $\text{CPI} = 1.1$ and $f = 2.4$ GHz. A CISC-like implementation executes 0.80×10^9 instructions at $\text{CPI} = 2.0$ and $f = 3.0$ GHz.

$$T_R = \frac{1.20 \times 10^9 \times 1.1}{2.4 \times 10^9} = 0.55 \text{ s},$$
$$T_C = \frac{0.80 \times 10^9 \times 2.0}{3.0 \times 10^9} \approx 0.533 \text{ s}.$$

- ▶ The CISC-like system wins this workload by about 3.0% despite its higher CPI, because it executes fewer instructions at a higher clock.
- ▶ Change the workload, cache behavior, or branch rate and the result can change. The measurement is the conclusion; the labels are hypotheses.

Pipelining Meets the ISA Boundary

Pipeline concern	Regular instruction stream helps	Rich instruction stream may require
Fetch/decode	Predictable boundaries and widths	Boundary detection and wider decode
Data hazards	Explicit register dependencies	Internal micro-op dependencies
Structural hazards	Regular stage/resource demand	Variable resource demand per instruction
Control recovery	Explicit branch behavior	Precise recovery across expanded work

Course-level connection

Week 5's addressing modes, Weeks 6–7's control units, Week 9–10's memory hierarchy, and Week 11's hazards all reappear here as constraints on instruction throughput.

The final design question is always: fit to which workload?

A Single Performance Thread Through CS401

1. **Representation:** bits, two's complement, and IEEE 754 determine what values the hardware can encode.
2. **Execution:** registers, buses, addressing modes, and control units move and transform those values.
3. **Communication:** I/O and interrupts connect the processor to devices without making every transfer a busy wait.
4. **Storage:** caches, virtual memory, and secondary storage trade capacity, latency, protection, and persistence.
5. **Overlap:** parallel organizations and pipelines raise throughput when hazards are identified and paid for explicitly.
6. **Choice:** the ISA and organization should be judged by the workload's measured time, energy, reliability, and software constraints.

Doubt-Clearing: The Six Questions to Ask

When a problem looks unfamiliar

1. What is the representation: bits, fields, signedness, or format?
2. Where is the operand: register, memory, cache, page, or device?
3. Which hardware state changes: PC, IR, MAR/MDR, register, flag, or control word?
4. What is the bottleneck: clock, CPI, bus, memory, I/O, or hazard?
5. What is the unit of comparison: latency, throughput, speedup, hit rate, miss penalty, or total CPU time?
6. Which assumption makes the shortcut valid, and what breaks it?

A strong exam solution names the model before doing arithmetic.

Socratic Check: Final Integrator

Scenario

A workload has many independent vector operations, frequent branches, and a large working set. It runs on a multicore processor with a pipelined front end. Which earlier course topics should you inspect first?

- ▶ **Week 11 / Week 12:** SIMD or MIMD opportunity, pipeline hazards, branch penalty, and instruction-stream choice.
- ▶ **Weeks 9–10:** cache locality, TLB/page behavior, and storage misses that can dominate an otherwise fast pipeline.
- ▶ **Week 1:** total time through $IC \times CPI/f$; never infer performance from clock rate alone.

Exam and Assignment Review Checklist

- ▶ **Performance:** write the equation, normalize units, and compare total time or speedup.
- ▶ **Instruction cycle/control:** name the state and write RTL transfers with PC, IR, MAR, and MDR.
- ▶ **Memory:** split addresses into tag/index/offset or page/offset before translating or computing a hit.
- ▶ **I/O:** identify who waits, who owns the bus, and where the data moves.
- ▶ **Pipeline:** draw a cycle table; classify each hazard; count every stall and flush.
- ▶ **RISC/CISC:** state the workload, measured variables, and the trade-off before defending a design choice.

Closing the Arc: From Bits to Design Judgment

The course's recurring question

How can a finite amount of hardware keep the common work moving quickly, correctly, and predictably?

- ▶ Representation makes operations possible.
- ▶ Organization moves operands and generates control.
- ▶ Hierarchy hides slow capacity behind fast common cases.
- ▶ Parallelism overlaps work, while hazards expose the dependencies.
- ▶ RISC/CISC choices shape the instruction work; the workload decides whether the trade-off pays.

Final takeaway

Do not ask which architecture is universally best. Ask which design makes this workload's useful work per unit time highest, and show the assumptions that make your answer true.

References and Source Boundaries

- ▶ Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th ed. (repository key retained as Stallings2015): Ch. 16 §16.4, “Instruction Pipelining” (PDF pp. 667–685); Ch. 17, especially §17.5 (PDF p. 730) and §17.9 (PDF p. 752); Ch. 20 §20.1, “Multiple Processor Organizations” (PDF p. 847).
- ▶ Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, ARM 5e: Ch. 4 §§4.5–4.8 (pp. 283, 297, 316, 328); Ch. 6 §§6.2–6.3 (pp. 518, 523); App. D (pp. D-2–D-43).
- ▶ The worked comparisons in this deck are constructed examples. Their numbers illustrate the performance equation and pipeline model; they are not quotations or measurements from either book.

Bibliography I



David A. Patterson and John L. Hennessy.

Computer Organization and Design: The Hardware/Software Interface.

Morgan Kaufmann, arm edition (5th) edition, 2017.